

CPU Scheduling Algorithm:

CPU Scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated first to the CPU.

Terminologies Used in CPU Scheduling

- **Arrival Time:** The time at which the process arrives in the ready queue.
- **Completion Time:** The time at which the process completes its execution.
- **Burst Time:** Time required by a process for CPU execution.
- **Turn Around Time:** Time Difference between completion time and arrival time.

$$\text{Turn Around Time} = \text{Completion Time} - \text{Arrival Time}$$

- **Waiting Time(W.T):** Time Difference between turn around time and burst time.

$$\text{Waiting Time} = \text{Turn Around Time} - \text{Burst Time}$$

There are four types of CPU scheduling that exist.

1. **First Come, First Served Scheduling (FCFS) Algorithm:** This is the simplest CPU scheduling algorithm. In this scheme, the process which requests the CPU first, that is allocated to the CPU first. The implementation of the FCFS algorithm is easily managed with a FIFO queue. When a process enters the ready queue its PCB is linked onto the rear of the queue. The average waiting time under FCFS policy is quite long. Consider the following example:

Process	CPU time
P ₁	3
P ₂	5
P ₃	2
P ₄	4

Using FCFS algorithm find the average waiting time and average turnaround time if the order is P₁, P₂, P₃, P₄.

Solution: If the process arrived in the order P₁, P₂, P₃, P₄ then according to the FCFS the Gantt chart will be:

P ₁	P ₂	P ₃	P ₄	
0	3	8	10	14

The waiting time for process P₁ = 0, P₂ = 3, P₃ = 8, P₄ = 10 then the turnaround time for process P₁ = 0 + 3 = 3, P₂ = 3 + 5 = 8, P₃ = 8 + 2 = 10, P₄ = 10 + 4 = 14.

Then average waiting time = $(0 + 3 + 8 + 10)/4 = 21/4 = 5.25$

Average turnaround time = $(3 + 8 + 10 + 14)/4 = 35/4 = 8.75$

The FCFS algorithm is non preemptive means once the CPU has been allocated to a process then the process keeps the CPU until the release the CPU either by terminating or requesting I/O.

2. **Shortest Job First Scheduling (SJF) Algorithm:** This algorithm associates with each process if the CPU is available. This scheduling is also known as shortest next CPU burst, because the scheduling is done by examining the length of the next CPU burst of the process rather than its total length. Consider the following example:

Process	CPU time
P ₁	3
P ₂	5
P ₃	2
P ₄	4

Solution: According to the SJF the Gantt chart will be

P ₃	P ₁	P ₂	P ₄	
0	2	5	9	14

The waiting time for process P₁ = 0, P₂ = 2, P₃ = 5, P₄ = 9 then the turnaround time for process P₃ = 0 + 2 = 2, P₁ = 2 + 3 = 5, P₄ = 5 + 4 = 9, P₂ = 9 + 5 = 14.

Then average waiting time = $(0 + 2 + 5 + 9)/4 = 16/4 = 4$

Average turnaround time = $(2 + 5 + 9 + 14)/4 = 30/4 = 7.5$

The SJF algorithm may be either preemptive or non preemptive algorithm. The preemptive SJF is also known as shortest remaining time first.

Consider the following example.

Process	Arrival Time	CPU time
P ₁	0	8
P ₂	1	4
P ₃	2	9
P ₄	3	5

In this case the Gantt chart will be

P ₁	P ₂	P ₄	P ₁	P ₃	
0	1	5	10	17	26

The waiting time for process

$$P_1 = 10 - 1 = 9$$

$$P_2 = 1 - 1 = 0$$

$$P_3 = 17 - 2 = 15$$

$$P_4 = 5 - 3 = 2$$

The average waiting time = $(9 + 0 + 15 + 2)/4 = 26/4 = 6.5$

3. **Priority Scheduling Algorithm:** In this scheduling a priority is associated with each process and the CPU is allocated to the process with the highest priority. Equal priority processes are scheduled in FCFS manner. Consider the following example:

Process	Arrival Time	CPU time
P ₁	10	3
P ₂	1	1
P ₃	2	3
P ₄	1	4
P ₅	5	2

According to the priority scheduling the Gantt chart will be

P ₂	P ₅	P ₁	P ₃	P ₄	
0	1	6	16	18	19

The waiting time for process

$$P_1 = 6$$

$$P_2 = 0$$

$$P_3 = 16$$

$$P_4 = 18$$

$$P_5 = 1$$

$$\text{The average waiting time} = (0 + 1 + 6 + 16 + 18)/5 = 41/5 = 8.2$$

4. **Round Robin Scheduling Algorithm:** This type of algorithm is designed only for the time sharing system. It is similar to FCFS scheduling with preemption condition to switch between processes. A small unit of time called quantum time or time slice is used to switch between the processes. The average waiting time under the round robin policy is quite long. Consider the following example:

Process	CPU time
P ₁	3
P ₂	5
P ₃	2
P ₄	4

Time Slice = 1 millisecond.

P ₁	P ₂	P ₃	P ₄	P ₁	P ₂	P ₃	P ₄	P ₁	P ₂	P ₄	P ₂	P ₄	P ₂	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

The waiting time for process

$$P_1 = 0 + (4 - 1) + (8 - 5) = 0 + 3 + 3 = 6$$

$$P_2 = 1 + (5 - 2) + (9 - 6) + (11 - 10) + (12 - 11) + (13 - 12) = 1 + 3 + 3 + 1 + 1 + 1 = 10$$

$$P_3 = 2 + (6 - 3) = 2 + 3 = 5$$

$$P_4 = 3 + (7 - 4) + (10 - 8) + (12 - 11) = 3 + 3 + 2 + 1 = 9$$

The average waiting time = $(6 + 10 + 5 + 9)/4 = 7.5$

- Each process gets a small unit of CPU time (time quantum q), usually 10-100 milliseconds.
- After this time has elapsed, the process is preempted and added to the end of the ready queue.

Timer interrupts occur every quantum to trigger preemption + scheduling of next process.

If there are n processes in the ready queue and the time quantum is q , then:

- Each process gets $\frac{1}{n}$ of the CPU time, in chunks of at most q time units at once.
- No process waits more than $(n - 1)q$ time units.

Performances depend heavily on q

- **q too large** $\rightarrow$ degenerates to FCFS scheduling
- **q too small** $\rightarrow$ lot of time wasted in context switches, instead of process work